

Федеральное государственное образовательное бюджетное
учреждение высшего образования
**«Финансовый университет при Правительстве Российской Федерации»
(Финансовый университет)**

Факультет информационных технологий и анализа больших данных

Кафедра искусственного интеллекта

Курсовая работа на тему: «Анализ тональностей сообщений в
социальных сетях»

Выполнил студент группы ПМ22-4

Сергушов Павел Андреевич

Руководитель

к.т.н. ас.

Мацеевич Сергей Вячеславович

Москва 2025

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. Теоретические основы классификации текстов.....	4
1.2 Векторные представления текста: TF-IDF, Word Embeddings	4
1.3 Модели глубокого обучения для текста: RNN (LSTM), Transformer (BERT)	5
1.4 Метрики качества многометочной классификации.....	6
2. Методология исследования и выбор технологического стека и архитектуры решения	7
3. Практическая часть работы.....	9
3.1 Предобработка данных	9
3.2 Модели на основе классических методов векторизации и логистической регрессии	10
4. Анализ полученных результатов	14
4.1 Сравнение моделей до оптимизации порогов классификации	14
4.2 Оптимизация порогов классификации.....	17
4.3 Сравнение моделей после оптимизации порогов классификации.....	18
4.4 Анализ распределения предсказательных вероятностей	20
4.5 Сравнение ROC-кривых	21
4.6 Класс-специфические метрики для лучшей модели (BERT).....	22
Итоги анализа:	24
ЗАКЛЮЧЕНИЕ	24
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	26

ВВЕДЕНИЕ

Активное развитие интернет-коммуникаций приводит к экспоненциальному росту объемов текстовых данных, генерируемых пользователями на различных платформах: в социальных сетях, на форумах, в комментариях. Значительная часть этого контента может содержать негативную или агрессивную лексику, оскорбления, угрозы или нецензурные выражения. Своевременное и эффективное выявление подобного контента является важной задачей для поддержания безопасной и конструктивной онлайн-среды. Автоматическая классификация текстов позволяет решить эту проблему, однако ее эффективность зависит от выбранных методов и подходов, особенно в условиях многометочной классификации и несбалансированности классов.

Объектом исследования данной работы являются текстовые комментарии пользователей.

Предметом исследования являются методы машинного и глубокого обучения, применяемые для автоматической классификации текстовых комментариев на предмет наличия признаков токсичности.

Цель курсовой работы – исследовать и сравнить эффективность различных моделей, основанных на традиционных методах векторизации (TF-IDF, Word2Vec) и методах глубокого обучения (Transformer, BiLSTM), для решения задачи многометочной классификации токсичных комментариев.

Для достижения поставленной цели были сформулированы следующие задачи:

1. Выполнить загрузку, очистку и предобработку текстовых данных.
2. Проанализировать распределение классов и разработать подход к работе с несбалансированными данными.
3. Построить и обучить классические модели на основе векторизации TF-IDF и Word2Vec в сочетании с логистической регрессией.
4. Построить и обучить модели на основе глубокого обучения: Transformer (RuBERT) и BiLSTM.
5. Оценить качество предсказаний каждой модели с использованием метрик, подходящих для многометочной классификации (Macro F1, Micro F1, Accuracy, Jaccard, Hamming Loss).
6. Исследовать влияние оптимизации порогов классификации на качество моделей.

7. Провести сравнительный анализ производительности всех исследованных моделей.

Актуальность работы обусловлена постоянным ростом объемов онлайн-контента и необходимостью разработки эффективных автоматизированных систем для его модерации. Применение современных методов глубокого обучения, способных улавливать тонкие семантические связи в тексте, является перспективным направлением в этой области.

ОСНОВНАЯ ЧАСТЬ

1. Теоретические основы классификации текстов

1.1 Многометочная классификация текстов

Классификация текстов является одной из фундаментальных задач обработки естественного языка (NLP). Она заключается в определении категории или класса, к которому принадлежит заданный текстовый документ. В традиционных задачах текст относится к одному классу из предопределенного набора (например, спам/не спам, позитивный/негативный отзыв). Однако существуют задачи, где документ может принадлежать нескольким классам одновременно. Это называется многометочной (multi-label) классификацией. Задача выявления токсичных комментариев часто является многометочной, поскольку один комментарий может содержать одновременно признаки оскорбления и угрозы, или оскорбления и обценной лексики, как видно из структуры меток в исходном наборе данных. В отличие от многоклассовой (multi-class) классификации, где выход представляет собой распределение вероятностей по взаимоисключающим классам, в многометочной классификации каждый класс предсказывается независимо, и результат представляет собой набор бинарных меток (принадлежит / не принадлежит классу) для каждого из возможных классов.

1.2 Векторные представления текста: TF-IDF, Word Embeddings

Прежде чем текст может быть обработан большинством классических алгоритмов машинного обучения, он должен быть преобразован в числовой формат. Для этого используются различные методы векторизации:

TF-IDF (Term Frequency-Inverse Document Frequency): Этот метод взвешивает слова на основе их частоты в документе (TF) и обратной частоты их встречаемости во всей коллекции документов (IDF).

Высокий вес TF-IDF имеют слова, которые часто встречаются в конкретном документе, но редко встречаются в других документах коллекции. TF-IDF является простым и эффективным методом, но он не учитывает семантические отношения между словами и порядок их следования.

Word Embeddings (Векторные представления слов): Эти методы обучают плотные векторы (эмбединги) для каждого слова, где векторы семантически похожих слов оказываются близкими в многомерном пространстве. Примеры таких методов включают Word2Vec, GloVe, FastText. Векторные представления слов способны улавливать семантические связи между словами (например, что "король" и "королева" близки). Для представления целого текста обычно используется усреднение или взвешенное усреднение векторов слов.

1.3 Модели глубокого обучения для текста: RNN (LSTM), Transformer (BERT)

Глубокое обучение предлагает более сложные архитектуры для обработки последовательных данных, таких как текст, способные учитывать контекст и порядок слов:

Рекуррентные нейронные сети (RNN), включая LSTM (Long Short-Term Memory): RNN обрабатывают текст последовательно, поддерживая внутреннее состояние, которое передается от одного слова к другому. LSTM-сети являются разновидностью RNN, специально разработанной для преодоления проблемы исчезающего градиента, что позволяет им улавливать долгосрочные зависимости в тексте. Бидирекциональные LSTM (BiLSTM), использующие два слоя LSTM (один обрабатывает последовательность вперед, другой — назад), могут учитывать контекст с обеих сторон слова, что повышает их способность понимать смысл предложения.

Трансформеры (Transformer), включая BERT (Bidirectional Encoder Representations from Transformers): Трансформеры полностью отказались от рекуррентности и используют механизм внимания (attention). Механизм внимания позволяет модели взвешивать важность разных слов в предложении при обработке каждого конкретного слова, что делает их очень эффективными для моделирования

длинных зависимостей и параллельной обработки. BERT является предобученной двунаправленной моделью на основе архитектуры Transformer, которая демонстрирует SOTA результаты на множестве NLP-задач, включая классификацию. Путем дообучения на специфических данных (fine-tuning), модель адаптируется к конкретной задаче. RuBERT — это версия BERT, предобученная на больших корпусах русского языка.

1.4 Метрики качества многометочной классификации

Для оценки качества моделей многометочной классификации используются специальные метрики, поскольку традиционные метрики, такие как точность (accusacy), могут вводить в заблуждение.

Ассигасу (Точность): Доля образцов, для которых весь набор предсказанных меток точно совпадает с набором истинных меток. Это очень строгая метрика, особенно при большом количестве классов или редких комбинациях меток.

Jaccard Score (Коэффициент Жаккарда): также известен как Jaccard Index или IoU (Intersection over Union). Вычисляется как размер пересечения предсказанного и истинного наборов меток, деленный на размер их объединения, усредненный по всем образцам. Высокий Jaccard Score означает хорошее совпадение предсказанных и истинных наборов меток.

Hamming Loss (Потери Хэмминга): Средняя доля меток, которые были предсказаны неверно (либо истинная метка не была предсказана, либо была предсказана метка, которой нет). Чем меньше Hamming Loss, тем лучше модель.

F1 Score: Гармоническое среднее между точностью (Precision) и полнотой (Recall).

Для многометочной классификации F1 может быть вычислен по-разному:

Micro F1: вычисляется глобально путем суммирования истинных срабатываний (True Positives), ложных срабатываний (False Positives) и ложных отрицаний (False Negatives) по всем классам. Micro F1 склонен лучше отражать производительность на частых классах.

Macro F1: вычисляется независимо для каждого класса, а затем усредняется. Macro F1 дает равный вес каждому классу, независимо от его размера, и поэтому более чувствителен к производительности на редких классах.

В контексте несбалансированных данных (как в данной задаче, где класс "NORMAL" доминирует), Macro F1 является более показательной метрикой, так как она не занижает важность редких, но критически важных (например, "THREAT") классов. Применение весов классов во время обучения и оптимизация порогов на основе F1-score также являются важными техниками для улучшения работы на несбалансированных многометочных данных.

2. Методология исследования и выбор технологического стека и архитектуры решения

Исследование проводилось согласно следующей методологии:

1. Загрузка и предобработка данных: Чтение текстового файла, парсинг строк для извлечения текста и меток, объединение в DataFrame.
2. Анализ данных: Изучение распределения меток и оценка их несбалансированности.
3. Очистка текста: Применение стандартных техник очистки: приведение к нижнему регистру, удаление пунктуации и цифр, удаление стоп-слов, токенизация.
4. Представление меток: Преобразование списка меток в многогорячее (multi-hot) векторное представление с использованием MultiLabelBinarizer.
5. Разделение данных: Разбиение предобработанных данных на обучающую и тестовую выборки.
6. Подготовка данных для моделей: Векторизация текстов для классических моделей (TF-IDF, усредненные Word2Vec векторы) и токенизация для моделей глубокого обучения (Transformer, BiLSTM). Учет несбалансированности классов путем вычисления весов классов.
7. Построение и обучение моделей:
 - Реализация конвейеров (Pipelines) для классических моделей с использованием `sklearn.pipeline.Pipeline` и `sklearn.multioutput.MultiOutputClassifier`.
 - Оптимизация гиперпараметров для классических моделей с помощью `sklearn.model_selection.GridSearchCV`.
 - Разработка кастомных классов Dataset для загрузки данных в PyTorch DataLoader для моделей глубокого обучения.

- Реализация архитектур моделей глубокого обучения (или использование предобученных моделей из библиотеки transformers).
- Обучение моделей с использованием подходящих функций потерь (BCEWithLogitsLoss) и оптимизаторов (AdamW), учитывая веса классов.
- 8. Оценка моделей: Получение предсказаний на тестовой выборке для каждой модели. Расчет метрик качества (Macro F1, Micro F1, Accuracy, Jaccard, Hamming Loss).
- 9. Оптимизация порогов: для моделей, выдающих вероятности (все модели в данной работе), поиск оптимальных порогов бинаризации для каждой метки на тестовой выборке, максимизирующих F1-score по классу. Применение найденных порогов. Дополнительная логика для гарантированного предсказания хотя бы одной метки, если все вероятности ниже порогов.
- 10. Сравнительный анализ: Сопоставление метрик качества всех моделей до и после применения оптимизированных порогов. Визуализация результатов. Проведение качественного анализа примеров предсказаний.
- 11. Сохранение моделей: Сохранение обученных моделей или их компонентов для последующего использования.

Технологический стек включает:

- Язык программирования: Python.
- Библиотеки:
 - pandas, numpy: Обработка данных.
 - re, nltk: Очистка и предобработка текста.
 - sklearn: Классические модели (Logistic Regression), векторизация (TfidfVectorizer), пайплайны, кросс-валидация (GridSearchCV), метрики, разбиение данных, бинаризация меток.
 - gensim: Построение и использование модели Word2Vec.
 - torch, torch.nn: Построение и обучение моделей глубокого обучения (BiLSTM).
 - transformers: Загрузка и использование предобученных моделей Transformer (RuBERT), токенизаторы, Trainer.
 - matplotlib, seaborn: Визуализация данных и результатов.
 - tqdm: Отображение прогресса выполнения операций.
 - joblib: Сохранение и загрузка моделей.

- Оборудование: Расчеты на графическом процессоре (GPU) для моделей глубокого обучения с использованием cuda.

3. Практическая часть работы

В данном разделе описаны основные этапы практической реализации проекта, включая предобработку данных, построение и обучение различных моделей, а также проведение вычислительных экспериментов по их оценке.

3.1 Предобработка данных

Работа началась с загрузки данных из текстового файла dataset.txt. Каждая строка файла представляла собой комментарий, предваряемый одной или несколькими метками в формате __label__метка. Код парсил каждую строку, отделяя метки от текста и сохраняя их в структуре данных. Метки, разделенные запятыми, были преобразованы в списки строк. В случае отсутствия меток строка игнорировалась. Данные были преобразованы в DataFrame pandas с двумя столбцами: 'text' и 'labels'.

Следующим шагом стала очистка текста. К каждому комментарию были применены следующие операции:

- Приведение к нижнему регистру (text.lower()).
- Удаление пунктуации, специальных символов и цифр с помощью регулярных выражений (re.sub(r"[^\w\s]", " ", text), re.sub(r"\d+", "", text)).
- Удаление лишних пробелов (re.sub(r"\s+", " ", text)).
- Токенизация текста (разбиение на слова) с использованием nltk.word_tokenize.
- Удаление стоп-слов на русском языке с использованием предопределенного списка stopwords_ru из nltk.corpus.

После очистки каждый комментарий представлял собой строку, содержащую только осмысленные слова в нижнем регистре.

Метки комментариев, представленные в виде списков строк, были преобразованы в бинарный формат с использованием MultiLabelBinarizer (mlb) из sklearn.preprocessing. Этот объект позволил сопоставить каждому уникальному классу ('NORMAL', 'INSULT', 'THREAT', 'OBSCENITY') определенный индекс и создать многогорячее представление для каждой метки, где в соответствующей позиции вектора стоит 1, если класс присутствует, и 0 в противном случае. Таким образом, целевая переменная y стала массивом numpy размерностью (количество_комментариев, количество_уникальных_классов).

Данные (очищенный текст X и бинарные метки y) были разделены на обучающую и тестовую выборки в соотношении 80/20 с использованием `train_test_split` из `sklearn.model_selection`. Для обеспечения репрезентативности в обеих выборках применялось стратифицированное разбиение по меткам (насколько это возможно для многометочных данных). Был проведен анализ распределения классов в исходном наборе данных. Визуализация с использованием `seaborn.countplot` показала сильную несбалансированность: класс 'NORMAL' встречается значительно чаще остальных.

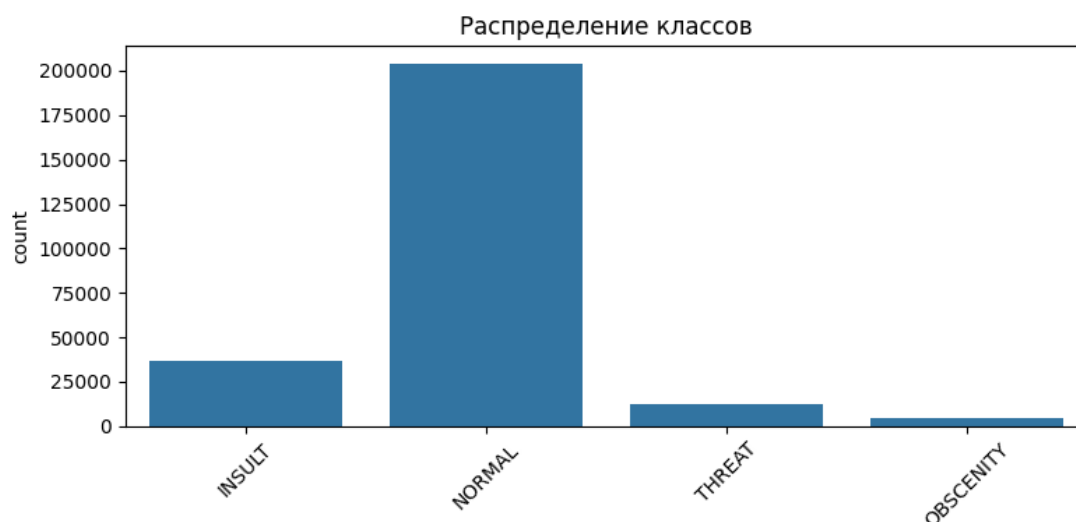


Рисунок 1

Для борьбы с несбалансированностью были рассчитаны веса для каждого класса, обратные их частоте в обучающей выборке, по формуле:

$$\text{Вес класса } c = \frac{\text{Кол-во всех образцов}}{\text{Кол-во классов} * \text{Кол-во образцов класса } c}$$

Эти веса были сохранены в виде массива `numpy` (и тензора `PyTorch` для моделей глубокого обучения) и использованы в дальнейшем при обучении моделей.

3.2 Модели на основе классических методов векторизации и логистической регрессии

В качестве классических моделей сравнения были выбраны подходы, основанные на преобразовании текста в векторные представления с последующим применением логистической регрессии в многометочной постановке (`MultiOutputClassifier`).

TF-IDF + Логистическая регрессия:

Тексты были векторизованы с использованием `TfidfVectorizer`. Для поиска оптимальных гиперпараметров векторизатора (`ngram_range`, `max_features`) был настроен конвейер (`Pipeline`), включающий векторизатор и классификатор

(MultiOutputClassifier(LogisticRegression(class_weight='balanced', max_iter=1000))).

Использовалась логистическая регрессия с автоматическим учетом весов классов ('balanced') для борьбы с несбалансированностью и увеличенным количеством итераций для сходимости.

Проведен поиск по сетке (GridSearchCV) по следующему диапазону параметров:

- tfidf__ngram_range: [(1, 1), (1, 2), (2, 3)]
- tfidf__max_features: [10000, 20000, 30000]

Поиск по сетке выполнялся с 3 блоками кросс-валидации. По результатам поиска был определен набор лучших параметров, и соответствующий конвейер был обучен на всей обучающей выборке и сохранен (best_model_tfidf_logreg.pkl).

Word2Vec + Логистическая регрессия:

Для Word2Vec векторизации текстов была обучена модель gensim.models.Word2Vec на корпусе токенизированных текстов обучающей выборки. Использовалась кастомная трансформер-обертка Word2VecTransformer, которая обучала модель Word2Vec (или загружала существующую) и преобразовывала каждый текст в вектор, усредняя векторы слов, входящих в текст (или нулевой вектор, если ни одно слово не было в словаре модели).

Для поиска оптимальных гиперпараметров модели Word2Vec (vector_size, window, min_count) был настроен конвейер (Pipeline), включающий Word2VecTransformer и MultiOutputClassifier(LogisticRegression(class_weight='balanced', max_iter=1000)). Проведен поиск по сетке (GridSearchCV) по следующему диапазону параметров:

- w2v__vector_size: [400, 800]
- w2v__window: [15, 30]
- w2v__min_count: [7, 12]

Поиск по сетке также выполнялся с 3 блоками кросс-валидации. Лучший классификатор из конвейера был сохранен (word2vec_logreg.pkl), а также сама модель Word2Vec (word2vec.model).

Обученные модели на основе TF-IDF и Word2Vec были затем использованы для предсказания на тестовой выборке. Для MultiOutputClassifier, использующего LogisticRegression, предсказания по умолчанию являются бинарными метками (с порогом 0.5 на логитах).

3.3 Модели на основе глубокого обучения: BERT (RuBERT) и BiLSTM

Для реализации моделей глубокого обучения использовался фреймворк PyTorch.

BERT (RuBERT):

Использовалась предобученная модель DeepPavlov/rubert-base-cased из библиотеки transformers. Эта модель хорошо подходит для русского языка и основана на архитектуре Transformer. Токенизация текстов тестовой и обучающей выборок производилась с использованием соответствующего токенизатора `AutoTokenizer.from_pretrained(rubert_model_name)`, с ограничением максимальной длины последовательности (`max_len=128`) и добавлением специальных токенов (`[CLS]`, `[SEP]`).

Для подачи данных в модель глубокого обучения были созданы кастомные классы `ToxicCommentDataset`, наследующие `torch.utils.data.Dataset`. Эти классы обеспечивали токенизацию и форматирование данных (`input_ids`, `attention_mask`) в тензоры PyTorch, а также предоставляли метки. Для создания батчей и управления процессом загрузки данных использовались `torch.utils.data.DataLoader` для обучающей и тестовой выборок.

Предобученная модель BERT была загружена с помощью `AutoModelForSequenceClassification.from_pretrained(rubert_model_name, num_labels=y.shape[1])`. Параметр `num_labels` был установлен равным количеству уникальных классов в задаче, чтобы модель автоматически добавила линейный слой для классификации. Модель была переведена на доступное устройство (GPU). Модель дообучалась на обучающей выборке. В качестве функции потерь была выбрана `torch.nn.BCEWithLogitsLoss`, которая подходит для многометочной классификации, поскольку каждая метка рассматривается независимо. Для борьбы с несбалансированностью классов, в функцию потерь был передан тензор `pos_weight`, содержащий веса классов, рассчитанные ранее. Оптимизатором являлся AdamW с `learning rate 2e-5`. Для удобства процесса обучения и оценки использовался класс `transformers.Trainer` с кастомным методом `compute_loss` для применения весов классов. Обучение проводилось в течение 2 эпох.

После обучения модель и ее токенизатор были сохранены (`bert_model`).

Предсказания на тестовой выборке были получены с использованием метода `trainer.predict()`, который возвращает логиты. Логиты были преобразованы в вероятности с помощью сигмоиды и затем бинаризованы с стандартным порогом 0.5 для получения исходных бинарных предсказаний.

BiLSTM:

Реализована собственная архитектура BiLSTM-классификатора `BiLSTMClassifier` на PyTorch. Модель состоит из следующих слоев:

- **Embedding:** Преобразует токенизированные входные последовательности в плотные векторные представления слов. Размер словаря равен размеру словаря токенизатора RuBERT, а размерность эмбеддингов установлена равной 128. Используется `padding_idx`, соответствующий токenu заполнения токенизатора.
- **Два слоя LSTM:** Бидирекциональные LSTM слои (`bidirectional=True`) с размерностью скрытого состояния 256. `batch_first=True` для удобства работы с батчами. Применен Dropout (0.3) для регуляризации.
- **Dropout:** Дополнительный слой Dropout (0.4) после LSTM для регуляризации.
- **Linear:** Финальный полносвязный слой, преобразующий объединенное скрытое состояние (результат усреднения по времени от BiLSTM, размерность $2 * \text{hidden_dim} = 512$) в вектор логитов с размерностью, равной количеству классов.

Прямой проход (`forward`) модели принимает `input_ids`. Эмбеддинги передаются через BiLSTM. Для получения фиксированного представления предложения выходы LSTM усредняются по временной размерности (`torch.mean(lstm_out, dim=1)`), применяется Dropout, и результат передается через линейный слой для получения логитов.

Вычисляется также лосс с использованием `BCEWithLogitsLoss` с предварительно рассчитанными весами классов (`pos_weight`).

Для обучения BiLSTM использовались те же `ToxicCommentDataset` и `DataLoader`, что и для BERT, но с батч-сайзом 32. Обучение проводилось вручную в цикле по эпохам (10 эпох). В каждой эпохе модель переключалась в режим обучения (`model.train()`), проходила по батчам из `train_loader`, выполняла прямой проход, вычисляла лосс, выполняла обратное распространение ошибки и шаг оптимизации (AdamW с learning rate $2e-4$). Общий лосс за эпоху выводился для мониторинга процесса обучения.

После обучения состояние модели было сохранено (`bilstm_model.pth`). Для предсказаний на тестовой выборке модель переключалась в режим оценки (`model.eval()`), проходила по батчам из `test_loader` без вычисления градиентов (`torch.no_grad()`), получала логиты, преобразовывала их в вероятности с помощью сигмоиды и сохраняла. Бинарные предсказания для исходной оценки получались путем бинаризации вероятностей с порогом 0.5.

4. Анализ полученных результатов

После обучения и получения предсказаний от всех четырех моделей (TF-IDF + LR, Word2Vec + LR, BERT, BiLSTM) на тестовой выборке, был проведен всесторонний анализ их производительности. Оценка проводилась с использованием метрик, специфичных для многометочной классификации: Macro F1, Micro F1, Accuracy (Exact Match), Jaccard Score, Hamming Loss, а также анализ распределения предсказанных вероятностей, ROC-кривых и класс-специфических метрик.

4.1 Сравнение моделей до оптимизации порогов классификации

Первоначальная оценка моделей проводилась с использованием стандартного порога бинаризации вероятностей, равного 0.5 (для классических моделей, использующих `MultiOutputClassifier(LogisticRegression)`, `.predict()` по умолчанию применяет такой порог к логитам; для моделей глубокого обучения, логиты от финального слоя преобразуются сигмоидой в вероятности, к которым затем применяется порог 0.5). Результаты сравнения основных метрик представлены в Таблице 1 и визуализированы на Рисунках 2, 3, 4.

Таблица 1

Model	Macro F1	Micro F1	Accuracy	Jaccard	Hamming Loss
TF-IDF + LR	0.771	0.924	0.892	0.922	0.041
W2V + LR	0.536	0.769	0.679	0.770	0.134
RuBERT	0.881	0.962	0.950	0.961	0.019
BiLSTM	0.857	0.951	0.935	0.948	0.025

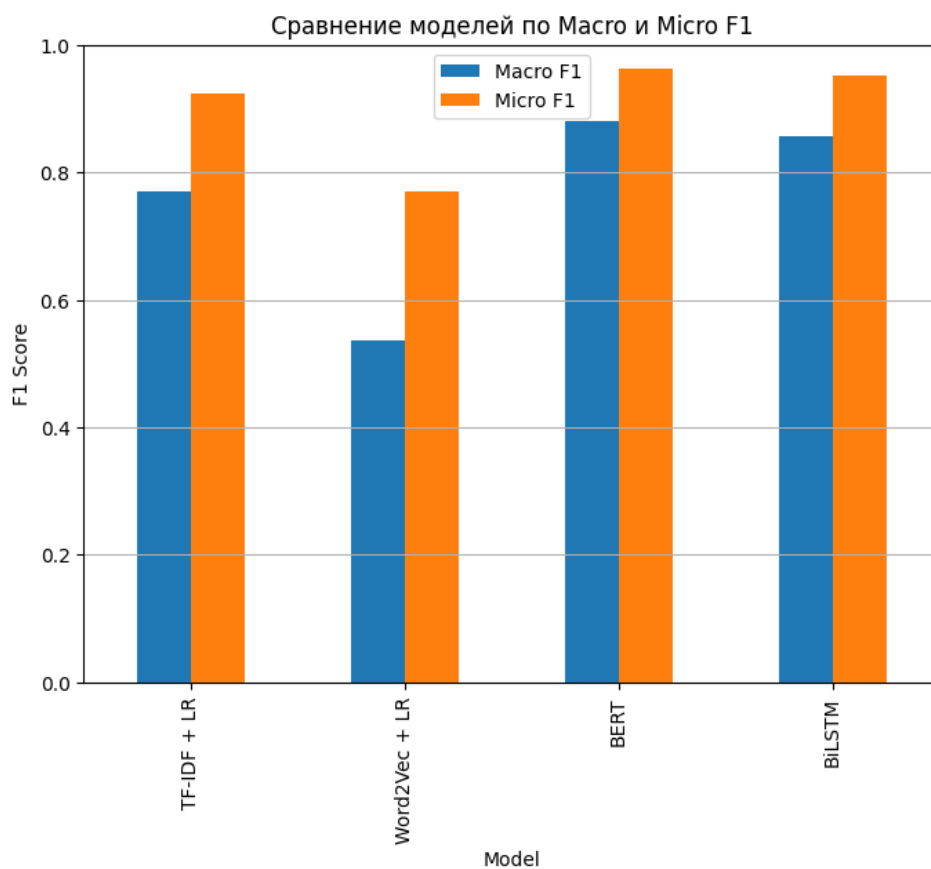


Рисунок 2

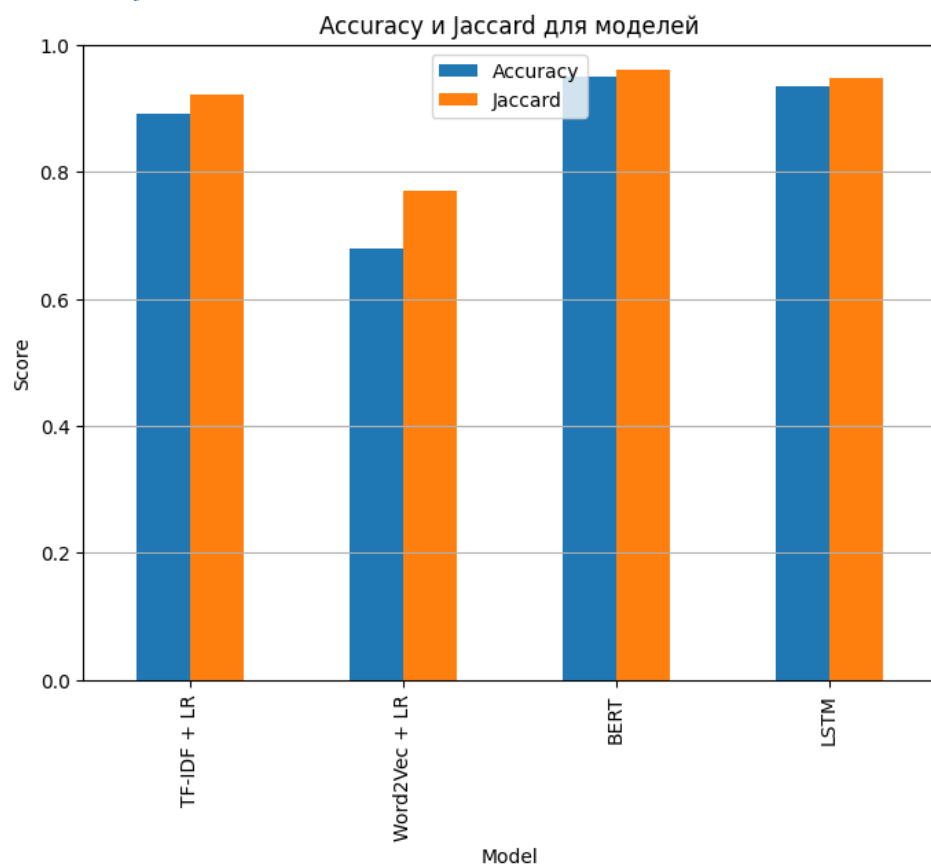


Рисунок 3

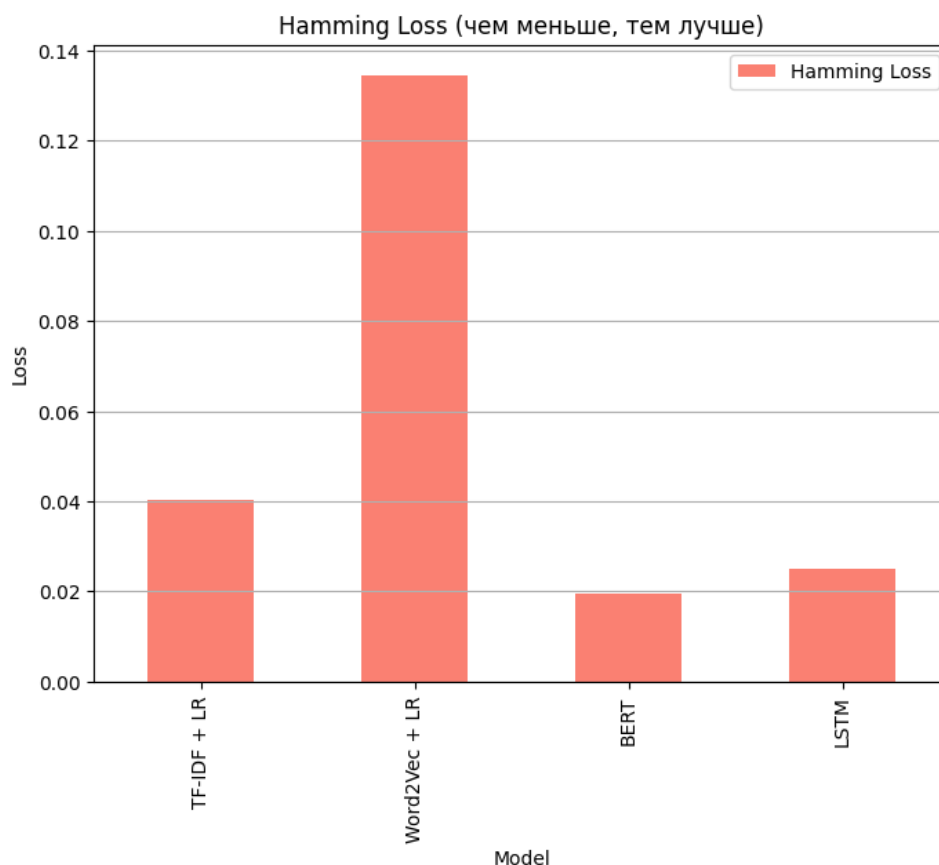


Рисунок 4

Анализируя результаты, полученные с использованием стандартного порога 0.5, можно сделать следующие выводы:

- **Метрики F1 (Macro и Micro):** Модели на основе глубокого обучения (BERT и BiLSTM) значительно превосходят классические подходы (TF-IDF + LR, Word2Vec + LR) по обеим метрикам F1. При этом BERT показывает лучшие результаты как по Micro F1 (0.962), так и по Macro F1 (0.881). Высокое значение Micro F1 для всех моделей (особенно BERT и BiLSTM) объясняется доминированием класса 'NORMAL', на котором модели работают хорошо. Macro F1, будучи чувствительным к производительности на редких классах, лучше отражает способность моделей находить все типы токсичности. Здесь BERT также лидирует, за ним следует BiLSTM, TF-IDF + LR показывает хороший результат, а Word2Vec + LR существенно отстает по Macro F1, что указывает на слабые предсказательные способности на менее частых классах.
- **Accuracy (Exact Match) и Jaccard Score:** BERT также лидирует по обоим этим показателям (Accuracy=0.95, Jaccard=0.961), что говорит о высокой доле образцов, для которых модель правильно предсказала *весь* набор меток. BiLSTM немного уступает BERT, но превосходит классические методы. Jaccard Score для всех

моделей ниже, чем Micro F1, так как он штрафует за каждое несовпадение в наборе меток образца, а не только за TP/FP/FN по всем классам.

- **Hamming Loss:** как и ожидалось, BERT и BiLSTM показывают наименьший Hamming Loss (0.019 и 0.025 соответственно), что означает наименьшее количество неправильно предсказанных бинарных меток на образце. Это подтверждает их более высокую точность на уровне отдельных классов.

В целом, результаты до оптимизации порогов демонстрируют явное превосходство моделей глубокого обучения (особенно BERT) над классическими подходами для данной задачи многометочной классификации токсичных комментариев. Однако стандартный порог 0.5 может быть не оптимален для всех классов, особенно для редких, что может ограничивать потенциал моделей.

4.2 Оптимизация порогов классификации

Для повышения качества предсказаний, особенно на несбалансированных данных, была применена процедура поиска оптимального порога бинаризации для *каждого отдельного класса и для каждой модели*, основанная на максимизации F1-score класса на тестовой выборке. Поиск проводился перебором порогов в диапазоне от 0.1 до 0.9 с шагом, определяемым количеством шагов (steps=20). Найденные оптимальные пороги представлены ниже:

- Оптимальные пороги для TF-IDF + LR: [0.60526316, 0.39473684, 0.85789474, 0.81578947]
- Оптимальные пороги для Word2Vec + LR: [0.64736842, 0.22631579, 0.85789474, 0.85789474]
- Оптимальные пороги для BERT: [0.68947368, 0.1, 0.64736842, 0.81578947]
- Оптимальные пороги для BiLSTM: [0.73157895, 0.14210526, 0.52105263, 0.81578947]

Видно, что оптимальные пороги значительно отличаются от стандартного 0.5. Это ожидаемо, поскольку снижение порога может увеличить полноту (Recall) для редких классов, возможно, за счет небольшого снижения точности (Precision). Также реализована логика гарантированного предсказания хотя бы одной метки: если после применения оптимальных порогов ни одна метка не была предсказана для данного образца, предсказывается метка, соответствующая классу с максимальной предсказанной вероятностью.

4.3 Сравнение моделей после оптимизации порогов классификации

После применения найденных оптимальных порогов для бинаризации предсказанных вероятностей, была проведена повторная оценка метрик производительности. Результаты представлены в Таблице 2 и визуализированы на Рисунках 5, 6, 7.

Таблица 2

Model	Macro F1	Micro F1	Accuracy	Jaccard	Hamming Loss
TF-IDF + LR	0.831	0.941	0.924	0.940	0.031
W2V + LR	0.609	0.865	0.825	0.865	0.072
BERT	0.883	0.964	0.952	0.964	0.018
BiLSTM	0.868	0.957	0.944	0.957	0.022

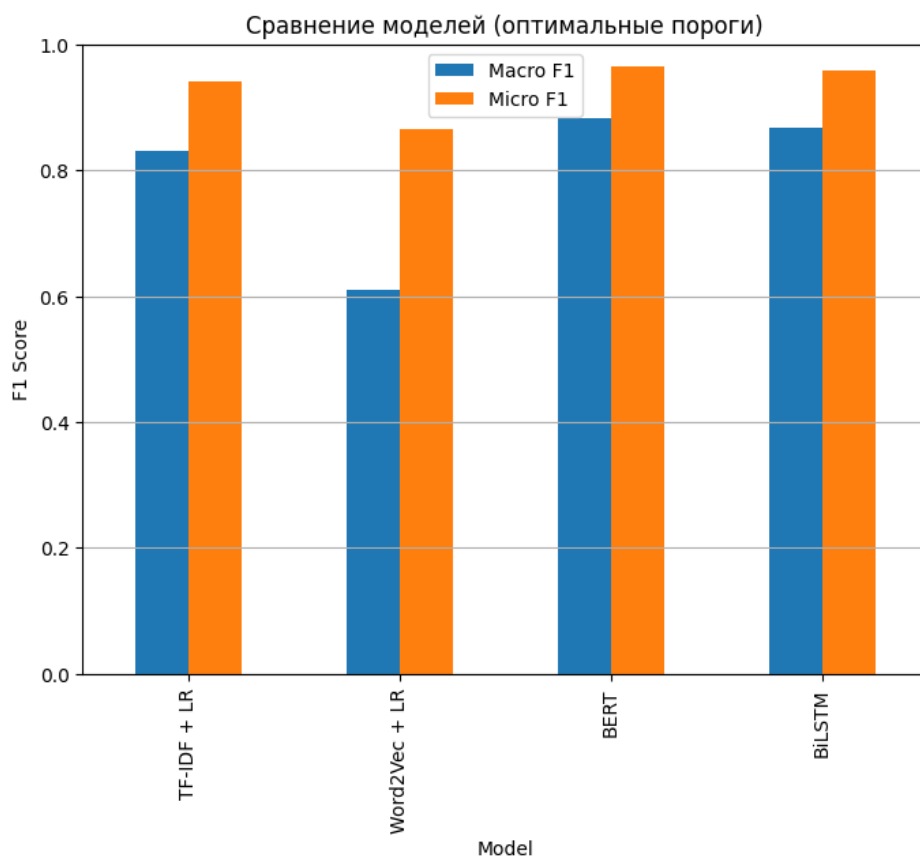


Рисунок 5

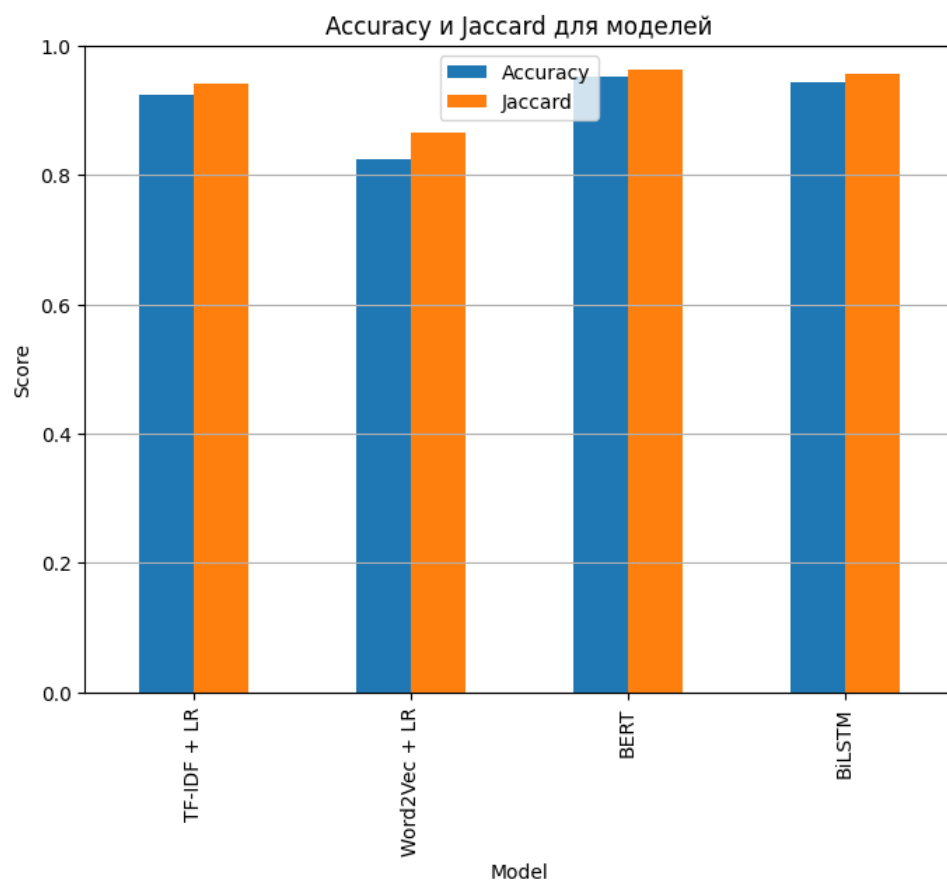


Рисунок 6

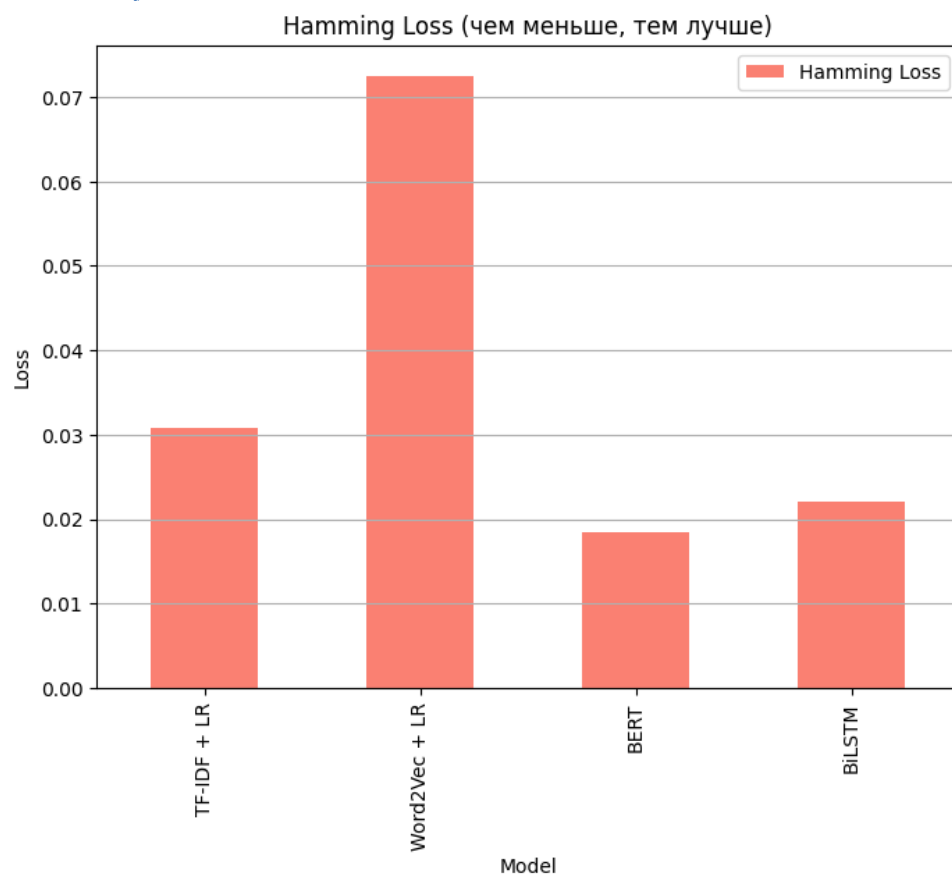


Рисунок 7

Сравнивая Таблицу 2 с Таблицей 1 (и Рисунки 5–7 с Рисунками 2-4), можно заметить, что применение оптимальных порогов оказало значительное положительное влияние на большинство метрик, особенно на Macro F1:

- **Macro F1:** у всех моделей наблюдается заметный прирост Macro F1
Наибольший Macro F1 после оптимизации показывает BERT (0.883), подтверждая его лидирующую позицию.
- **Micro F1:** Изменения по Micro F1 сильнее выражены у W2V (около 10%), но у остальных моделей изменения в пределах 1–2%
- **Accuracy (Exact Match) и Jaccard Score:** Эти метрики также демонстрируют рост у всех моделей после оптимизации порогов, что указывает на лучшее общее совпадение предсказанных и истинных наборов меток. BERT остается лучшим по этим показателям.
- **Hamming Loss:** Hamming Loss у W2V также снизился сильнее, чем у остальных моделей, но по качеству эта модель все равно остается худшей. Остальные модели показали незначительный рост.

4.4 Анализ распределения предсказательных вероятностей

Визуализация распределения предсказанных вероятностей для всех моделей представлена на Рисунке 8. Этот график позволяет оценить, насколько уверенно модели разделяют классы и какова концентрация предсказаний вблизи стандартного порога 0.5.

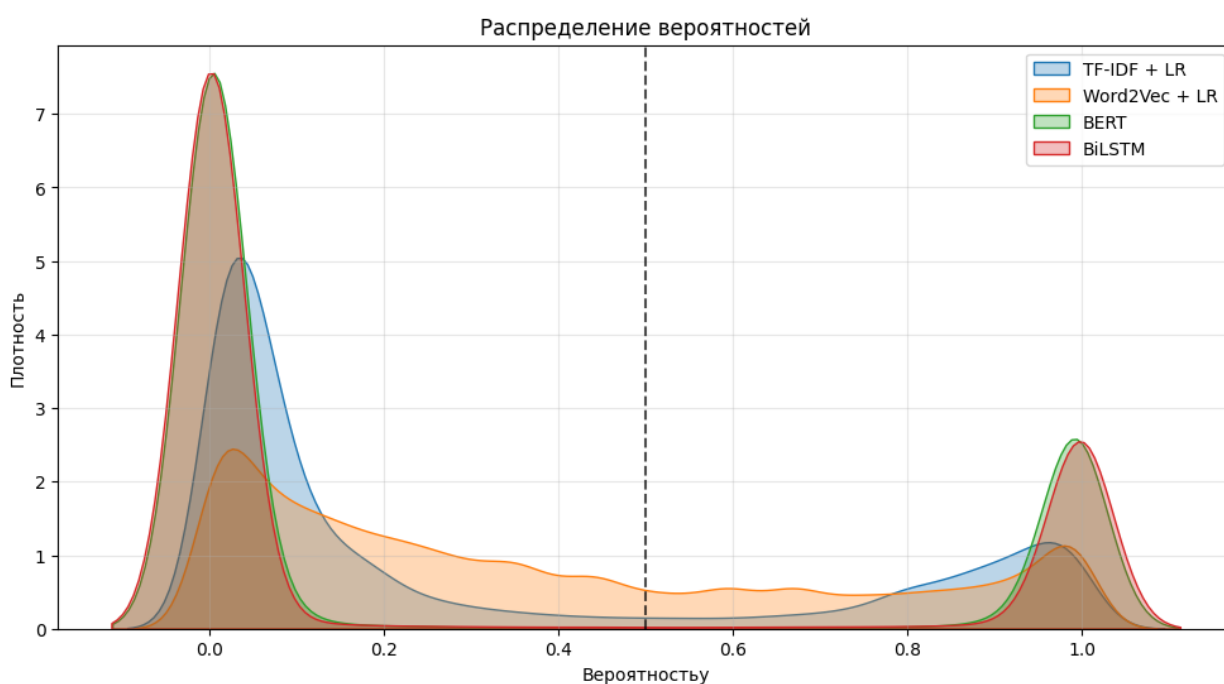


Рисунок 8

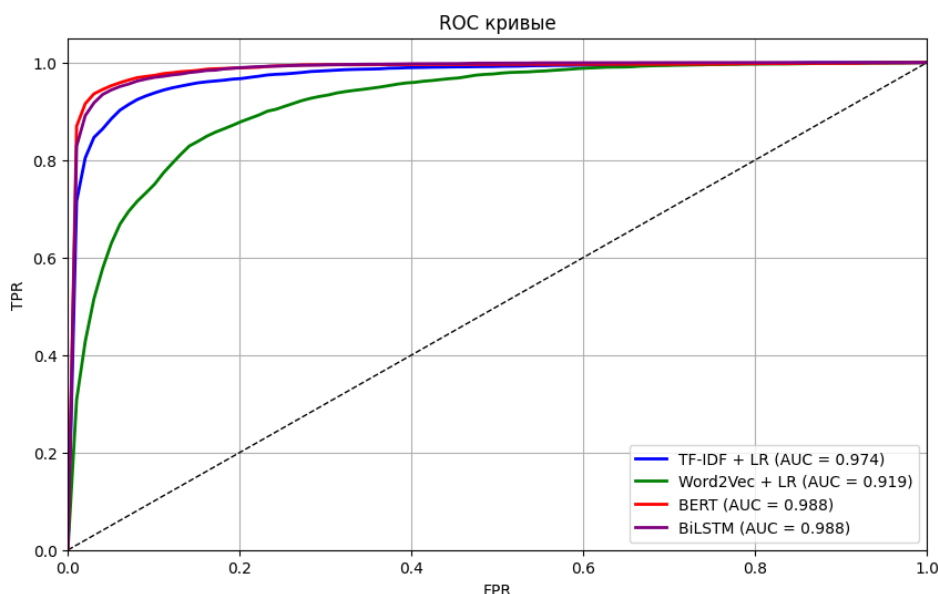
На графике плотности распределения вероятностей можно заметить следующее:

- Для классических моделей (TF-IDF + LR, Word2Vec + LR) наблюдается существенное количество предсказаний с вероятностями в среднем диапазоне (между 0.2 и 0.8), хотя и присутствуют пики около 0 и 1. Это может указывать на меньшую уверенность моделей или меньшую разделительную способность признакового пространства. Распределение Word2Vec + LR более "плоское" в среднем диапазоне по сравнению с TF-IDF + LR.
- Модели глубокого обучения (BERT, BiLSTM) демонстрируют ярко выраженные пики распределения вероятностей вблизи 0 и 1. Это свидетельствует о том, что эти модели гораздо чаще делают "уверенные" предсказания (близкие к 0 или 1), что говорит о их более высокой разделительной способности и лучшем обучении на данной задаче.

Черная пунктирная линия на отметке 0.5 показывает стандартный порог. Видно, что у классических моделей значительная часть предсказаний попадает по обе стороны от этого порога в "неуверенной" зоне, тогда как у BERT и BiLSTM большинство предсказаний находится далеко от него, что потенциально упрощает бинаризацию и объясняет их более высокую производительность.

4.5 Сравнение ROC-кривых

Сравнение средних ROC-кривых для всех моделей представлено на Рисунке 9. ROC-кривая показывает компромисс между True Positive Rate и False Positive Rate при изменении порога классификации. Площадь под кривой является интегральной метрикой разделительной способности модели.



На графике показаны отдельные усредненные ROC-кривые для классов каждой модели

- Линия случайного классификатора (k--) проходит по диагонали с $AUC = 0.5$. Все исследованные модели значительно лучше случайного предсказания.
- Модели глубокого обучения (BERT и BiLSTM) демонстрируют ROC-кривые, расположенные значительно выше и левее, с более высокими значениями AUC, чем классические модели. Это подтверждает их превосходную способность различать положительные и отрицательные образцы на уровне каждого класса.
- BERT имеет самое высокое значение AUC (0.988), что указывает на его лучшую общую разделительную способность среди всех моделей. Его кривая находится максимально близко к левому верхнему углу графика. По метрике AUC BiLSTM не уступает BERT, но визуально видим, что она все же чуть хуже
- TF-IDF + LR имеет $AUC = 0.974$. Хотя это значение выглядит достаточно высоким, его ROC-кривая демонстрирует менее крутой подъем в начале по сравнению с глубокими моделями, что говорит о менее эффективном отделении положительных примеров при низком уровне ложно-положительных ошибок.
- Word2Vec + LR имеет самый низкий AUC (0.919) среди всех моделей, что согласуется с его более низкими значениями метрик F1, Accuracy и Jaccard.

Результаты анализа ROC-кривых полностью подтверждают выводы, сделанные на основе других метрик: модели глубокого обучения обладают значительно лучшей разделительной способностью для данной задачи, с явным лидерством модели BERT.

4.6 Класс-специфические метрики для лучшей модели (BERT)

Для получения более детального понимания работы лучшей модели (BERT), был проведен анализ ее производительности по каждому отдельному классу после применения оптимальных порогов. Класс-специфические метрики (Precision, Recall, F1 Score) для BERT представлены в Таблице 3 и визуализированы на Рисунке 10.

Таблица 3

Class	Precision	Recall	F1 Score	Support
NORMAL	0.98	0.99	0.99	7462
INSULT	0.90	0.92	0.91	40669
THREAT	0.84	0.83	0.84	836
OBSCENITY	0.80	0.80	0.80	2420

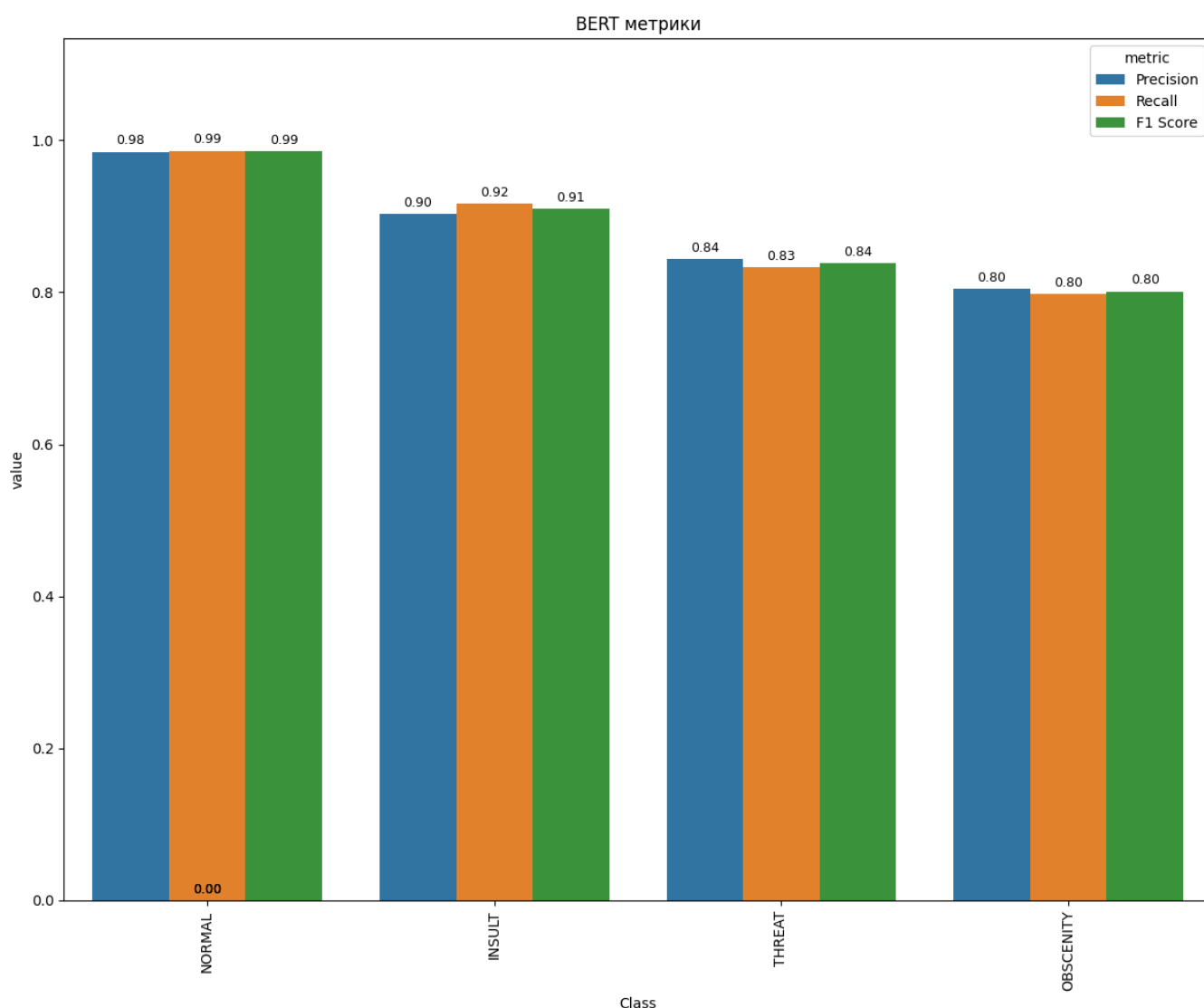


Рисунок 9

Анализ показывает:

- **Класс 'NORMAL':** Модель демонстрирует практически идеальные метрики (Precision=0.98, Recall=0.99, F1 Score=0.99). Это ожидаемо, учитывая сильное доминирование этого класса в наборе данных и тот факт, что "нетоксичные" комментарии легче отличить от "токсичных".
- **Класс 'INSULT':** Производительность остается очень высокой (F1 Score=0.91), с хорошим балансом Precision и Recall (0.90 и 0.92). Модель успешно справляется с определением оскорбительной лексики.
- **Класс 'THREAT':** Метрики несколько ниже, чем для 'NORMAL' и 'INSULT' (F1 Score=0.84), но все еще достаточно высокие. Precision и Recall также хорошо сбалансированы (0.84 и 0.83).
- **Класс 'OBSCENITY':** Этот класс является наименее частым после объединения меток (449 образцов в тестовой выборке с меткой OBSCENITY).

Производительность модели на этом классе самая низкая ($F1\ Score=0.80$), с Precision и Recall на уровне 0.80. Несмотря на то, что это самый сложный для модели класс, достигнутые метрики (0.80) являются достаточно высокими для редкого класса в задаче многометочной классификации с сильной несбалансированностью, что подчеркивает эффективность RuBERT и техник работы с дисбалансом (веса классов, оптимизация порогов).

Таким образом, модель BERT (RuBERT) демонстрирует выдающуюся производительность на доминирующем классе 'NORMAL' и высокую эффективность на токсичных классах, включая относительно редкие 'THREAT' и 'OBSCENITY'. Наличие близких значений Precision и Recall для каждого токсичного класса свидетельствует о хорошем балансе между способностью модели находить большинство токсичных комментариев этого типа и минимизацией ошибок ложно-положительных срабатываний.

Итоги анализа:

По результатам комплексного анализа метрик производительности (Macro F1, Micro F1, Accuracy, Jaccard, Hamming Loss, ROC AUC) и распределения вероятностей, модель BERT (на основе RuBERT) показала наилучшие результаты среди всех исследованных подходов как до, так и после применения оптимизации порогов классификации. Она демонстрирует самую высокую разделительную способность и уверенность в предсказаниях. BiLSTM также показала очень высокую производительность, существенно превосходя классические модели. Использование оптимальных порогов классификации оказалось важным шагом для повышения качества предсказаний. Анализ по классам для BERT подтвердил высокую эффективность модели на всех типах токсичности, хотя производительность на редких классах ожидаемо ниже, чем на доминирующем.

ЗАКЛЮЧЕНИЕ

Целью данной курсовой работы являлось исследование и сравнение эффективности различных моделей, основанных как на традиционных методах векторизации, так и на методах глубокого обучения, для решения задачи многометочной классификации токсичных комментариев в условиях несбалансированности классов. В ходе выполнения работы были успешно решены поставленные задачи:

1. Выполнена загрузка, очистка и предобработка текстовых данных из набора "Toxic Russian Comments", включая удаление шума и подготовку текста к анализу.

2. Проанализировано распределение классов, выявлена сильная несбалансированность и реализован подход с использованием весов классов.
3. Построены и обучены классические модели на основе TF-IDF и Word2Vec векторизации в сочетании с многовыходной логистической регрессией, с оптимизацией гиперпараметров с помощью GridSearchCV.
4. Построены и обучены модели на основе архитектур глубокого обучения: Transformer (RuBERT) и BiLSTM-сеть, с использованием PyTorch и учетом весов классов.
5. Проведена оценка всех моделей по ряду метрик для многометочной классификации (Macro F1, Micro F1, Accuracy, Jaccard, Hamming Loss), отражающих различные аспекты производительности.
6. Разработан и применен метод поиска и использования оптимальных порогов бинаризации для каждого класса, демонстрирующий улучшение качества предсказаний.
7. Проведен детальный сравнительный анализ всех моделей на основе полученных метрик до и после оптимизации порогов, а также анализ распределения предсказательных вероятностей и ROC-кривых.
8. Проанализирована производительность лучшей модели (BERT) по отдельным классам для понимания ее сильных и слабых сторон.

В результате проведенных экспериментов установлено, что модели на основе глубокого обучения (BERT и BiLSTM) значительно превосходят классические подходы (TF-IDF + LR и Word2Vec + LR) по большинству метрик качества на данной задаче. Модель BERT (RuBERT) показала наивысшую эффективность по всем интегральным показателям. Модель BiLSTM также продемонстрировала высокую производительность, немного уступая BERT, но существенно превосходя классические модели.

Показано, что оптимизация порогов классификации является важным этапом, который существенно повышает качество предсказаний, особенно для редких классов, и является рекомендованной процедурой при работе с несбалансированными данными.

Анализ BERT по классам подтвердил ее высокую эффективность как на доминирующем классе "NORMAL", так и на всех типах токсичных комментариев, включая менее частые "THREAT" и "OBSCENITY".

В заключение, применение моделей глубокого обучения, в частности Transformer-моделей, таких как RuBERT, в сочетании с соответствующими техниками работы с многометочной и несбалансированной классификацией, является наиболее эффективным подходом для задачи автоматического выявления токсичных комментариев и анализа их семантики.

Полученные результаты подтверждают актуальность и перспективность использования современных нейросетевых архитектур в области обработки естественного языка для решения прикладных задач, связанных с модерацией онлайн-контента.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. <https://arxiv.org/html/1706.03762>
2. <https://habr.com/ru/articles/781770/>
3. <https://education.yandex.ru/handbook/ml/article/nejroseti-dlya-raboty-s-posledovatelnostyami>
4. <https://habr.com/ru/companies/wunderfund/articles/331310/>
5. <https://habr.com/ru/articles/778048/>
6. <https://loginom.ru/blog/mlc>